RÉPUBLIQUE ALGÉRIENNE DÉMOCRATIQUE ET POPULAIRE
MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR ET DE LA RECHERCHE SCIENTIFIQUE
ÉCOLE NATIONALE SUPÉRIEURE D'INFORMATIQUE

Rapport de TP VCL N°6 2ème Année Cycle Supérieur (2CS)

Option : Systèmes Informatiques (SIQ)

Gestion d'applications conteneurisées avec Kubernetes

Réalisé par :

ATTIA Oussama Abderraouf
SRAICH Imene

Encadré par :

MENNACER Djamel-Eddine

Année universitaire : 2025–2026

Contents

Introduction **3**

1 Préparation de l'environnement **4**

1.1 Machine virtuelle Linux 4

1.2 Application conteneurisée sur Docker Hub 4

1.3 Installation des outils 4

2 Processus d'utilisation de Kubernetes CLI **5**

2.1 LAB 1 : Création d'un cluster Kubernetes 5

2.2 LAB 2 : Déploiement d'une application 5

2.3 LAB 3 : Exploration de l'application 5

2.4 LAB 4 : Publication de l'application 6

2.5 LAB 5 : Mise à l'échelle de l'application 6

2.6 LAB 6 : Mise à jour de l'application 7

3 Déploiement avec Kubernetes YAML **8**

Conclusion **9**

Introduction

Kubernetes (K8s) est un système open-source qui automatise le déploiement, la mise à l'échelle et la gestion des applications conteneurisées. C'est ce qu'on appelle l'orchestration de conteneurs. Pour ce TP, nous nous intéressons à **Minikube**, une implémentation légère de Kubernetes qui permet de faire tourner un cluster à un nœud unique sur une machine locale, idéal pour l'apprentissage et le développement.

À travers ce TP, nous allons suivre un processus complet de gestion d'une application conteneurisée avec Minikube, couvrant le déploiement, l'exploration, la publication, la mise à l'échelle et la mise à jour. Nous déploierons également la même application en utilisant une approche déclarative via un fichier de configuration YAML.

Chapter 1 : Préparation de l'environnement

1.1 Machine virtuelle Linux

Une machine virtuelle Ubuntu 20.04 est installée sur VMware Workstation. Elle sera le nœud unique de notre cluster Kubernetes.

Périphérique	**Configuration**
Mémoire (RAM)	3 GB
Processeurs	4
Disque Dur	20 GB
Réseau	NAT (pour l'accès à Internet)

1.2 Application conteneurisée sur Docker Hub

Nous utilisons l'image Docker créée dans le TP précédent (TP5b), publiée sur Docker Hub. Cette image contient une application web Python Flask.

- **Version 1 :** `samytadashi/tp5b-vcl:1`
- **Version 2 :** `souhailsadat/tp5b-vcl:2`

1.3 Installation des outils

- **Docker :** Nécessaire car Minikube l'utilisera comme driver de conteneur.
- **Minikube :** L'outil pour créer notre cluster local.
- **Kubectl :** L'interface en ligne de commande (CLI) pour interagir avec le cluster Kubernetes.

Les installations se font via les gestionnaires de paquets `apt` et `snap`.

Chapter 2 : Processus d'utilisation de Kubernetes CLI

2.1 LAB 1 : Création d'un cluster Kubernetes

```
# Lancement de Minikube
esi@ubuntu:~$ minikube start

# Verification des informations du cluster
esi@ubuntu:~$ kubectl cluster-info

# Affichage du noeud unique du cluster
esi@ubuntu:~$ kubectl get nodes

# Connexion a la console du noeud (un conteneur Docker)
esi@ubuntu:~$ minikube ssh
```

2.2 LAB 2 : Déploiement d'une application

```
# Creation du deploiement a partir de l'image Docker Hub (
   Version 1)
esi@ubuntu:~$ kubectl create deployment vcl-app --image=
   samytadashi/tp5b-vcl:1

# Lister les deploiements et les pods
esi@ubuntu:~$ kubectl get deployments
esi@ubuntu:~$ kubectl get pods

# Stocker le nom du pod dans une variable
esi@ubuntu:~$ export POD_NAME=$(kubectl get pods -o go-template
   --template '{{range .items}}{{.metadata.name}}{{end}}')
```

2.3 LAB 3 : Exploration de l'application

Pour accéder à l'application, nous utilisons un proxy qui relaie les requêtes depuis notre machine locale vers le réseau interne du cluster.

```
# Lancer le proxy dans un second terminal
esi@ubuntu:~$ kubectl proxy

# Interroger l'application via le proxy
esi@ubuntu:~$ curl http://localhost:8001/api/v1/namespaces/
   default/pods/$POD_NAME/proxy/
```

On peut également consulter les logs de l'application ou ouvrir un shell à l'intérieur du conteneur :

```
# Afficher les logs
esi@ubuntu:~$ kubectl logs $POD_NAME

# Executer une commande a l'interieur du conteneur
esi@ubuntu:~$ kubectl exec -ti $POD_NAME -- /bin/sh
```

2.4 LAB 4 : Publication de l'application

Pour exposer l'application à l'extérieur du cluster sans proxy, nous créons un service de type **NodePort**.

```
# Creer un service qui expose le deploiement
esi@ubuntu:~$ kubectl expose deployment vcl-app --type=NodePort
   --port=8080 --target-port=80

# Obtenir l'URL d'acces
esi@ubuntu:~$ minikube service vcl-app --url
```

L'application est maintenant accessible via l'IP de Minikube et un port externe attribué dynamiquement.

2.5 LAB 5 : Mise à l'échelle de l'application

Nous augmentons le nombre de répliques (pods) pour gérer plus de trafic.

```
# Mettre a l'echelle le deploiement a 4 repliques
esi@ubuntu:~$ kubectl scale deployments/vcl-app --replicas=4

# Verifier que les 4 pods sont en cours d'execution
esi@ubuntu:~$ kubectl get pods -o wide
```

2.6 LAB 6 : Mise à jour de l'application

Kubernetes permet des mises à jour sans interruption de service (**Rolling Update**).

```
# Mettre a jour l'image du deploiement vers la version 2
esi@ubuntu:~$ kubectl set image deployments/vcl-app vcl-app=
   souhailsadat/tp5b-vcl:2

# Suivre le statut du deploiement
esi@ubuntu:~$ kubectl rollout status deployments/vcl-app
```

Kubernetes va progressivement remplacer les anciens pods par des nouveaux exécutant la version 2 de l'application.

Chapter 3 : Déploiement avec Kubernetes YAML

L'approche déclarative via des fichiers YAML est la méthode recommandée pour gérer les applications sur Kubernetes. Créez un fichier `configuration.yaml` :

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: yaml-app
spec:
  selector:
    matchLabels:
      app: yaml-app
  replicas: 3
  template:
    metadata:
      labels:
        app: yaml-app
    spec:
      containers:
      - name: tp5b-vcl
        image: samytadashi/tp5b-vcl:1
        ports:
        - containerPort: 80
```

Appliquez cette configuration au cluster :

```
# Creer les objets definis dans le fichier YAML
esi@ubuntu:~$ kubectl create -f configuration.yaml

# Verifier les objets crees
esi@ubuntu:~$ kubectl get deployments
esi@ubuntu:~$ kubectl get pods
```

Un nouveau déploiement avec 3 pods a été créé, conformément au fichier YAML.

Conclusion

Ce TP nous a permis de découvrir l'écosystème Kubernetes à travers Minikube. Nous avons appris les commandes essentielles pour gérer le cycle de vie d'une application conteneurisée : déploiement, publication, mise à l'échelle et mise à jour. Nous avons également exploré les objets fondamentaux de Kubernetes (pods, services, déploiements) et compris l'avantage de l'approche déclarative avec les fichiers YAML pour une gestion d'infrastructure en tant que code.